

# Report Template

## FYS-STK3155 - Project X

Student  
*University of Oslo*  
(Dated: October 31, 2025)

This project investigates the implementation and performance of fully connected feedforward neural networks built entirely from scratch using NumPy. The goal is to develop an understanding of core neural network operations such as forward propagation, backpropagation, and gradient-based optimization. The network is first trained on the Runge function to evaluate its ability to approximate nonlinear functions and to study the influence of activation functions, learning rates, and network depth. While the neural network successfully captures the general shape of the Runge function, it performs worse than traditional polynomial regression methods in terms of mean squared error, mainly due to sensitivity to hyperparameter tuning and overfitting. In later tasks, the same implementation is applied to the MNIST handwritten digit dataset to assess its generalization capability on real-world classification data. Here, the model achieves approximately 93% accuracy, comparable to baseline models from standard machine learning libraries. Overall, the project bridges theoretical understanding and practical experimentation in neural network design and training, highlighting both the flexibility and the limitations of manual neural network implementations. (Note that the results are made up, as we have not finished the report yet)

### I. INTRODUCTION

Artificial neural networks (ANNs) have become a central tool in modern machine learning, capable of capturing complex nonlinear relationships between input and output data. They are widely used in applications such as image recognition, natural language processing, and biological data analysis. Despite their popularity, much of the underlying mechanics is hidden when using high-level libraries like TensorFlow, PyTorch, or scikit-learn. Understanding how neural networks operate internally is therefore essential for developing an intuition about their strengths, limitations, and numerical behavior.

In this project, we implement a fully connected feedforward neural network entirely from scratch using NumPy. By doing so, we aim to gain a detailed understanding of the main computational steps involved: the forward pass for generating predictions, the backward pass for computing gradients through backpropagation, and the parameter updates performed by gradient descent. This manual implementation allows us to explore how fundamental design choices—such as network depth, activation functions, and learning rate—affect model training and performance.

The project is divided into two main parts. In the first part, the implemented neural network is applied to a regression task using the Runge function. This provides a simple and controlled setting for studying how well the network approximates nonlinear functions, as well as how it behaves under different hyperparameter configurations. In the second part, we test the same implementation on the MNIST dataset of handwritten digits to evaluate its performance on real-world classification data. Comparing these results with those from standard machine learning libraries serves to validate our implementation and to highlight the trade-offs between flexibility, computational efficiency, and accuracy.

Through these experiments, the project aims to connect theoretical understanding with practical implementation. Building neural networks from first principles helps solidify the mathematical intuition behind gradient-based learning and clarifies how these concepts are realized in code.

### II. METHODS

#### A. Method 1/X

- Describe the methods and algorithms, including the motivation for using them and their applicability to the problem
- Derive central equations when appropriate, the text is the most important part, not the equations.

I used quite a lot of chatGPT on this report. From fine tuning my NN code to work properly for the IRIS data. To making a loop to calculate loss for all the NN variants. And plot their loss in the heatmap. As well as making the plots for the epoch. And helping me work with latex. And helping me write the abstract and introduction (Not from scratch, i went through a few iterations).

#### B. Implementation

- Explain how you implemented the methods and also say something about the structure of your algorithm and present very central parts of your code, not more than 10 lines
- You should plug in some calculations to demonstrate your code, such as selected runs used to validate and verify your results. A reader needs to un-

derstand that your code reproduces selected benchmarks and reproduces previous results, either numerical and/or well-known closed form expressions.

### C. Use of AI tools

- Describe how AI tools like ChatGPT were used in the production of the code and report.

## III. RESULTS AND DISCUSSION

This section presents the results from training and evaluating the neural network on the Iris dataset. Two key figures are included: one showing the training loss over epochs, and one visualizing test accuracy for different network architectures.

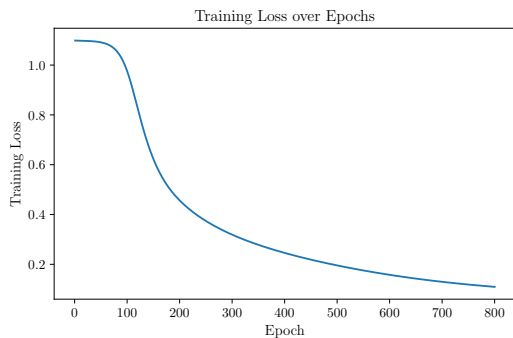


Figure 1: Training loss as a function of epochs for a fully connected neural network trained on the Iris dataset. The model consists of one hidden layer with 16 nodes and uses the ReLU activation function in the hidden layer and a softmax output layer. The loss decreases steadily over 800 epochs, indicating convergence of the model through gradient-based optimization.

The training loss decreases smoothly throughout the 800 epochs, showing that the network converges properly under the chosen learning rate and architecture. The relatively stable loss curve indicates that the gradient updates are well-behaved and that the model efficiently learns to minimize the cross-entropy loss. This suggests that the chosen architecture, consisting of a single hidden layer with 16 nodes, is sufficient to capture the underlying structure of the Iris dataset without overfitting.

The heatmap 2 shows that models with zero or one hidden layer achieve the highest test accuracy, around 96–97 percent, while deeper networks with two or three hidden layers perform worse. This outcome is expected for the Iris dataset, which is almost linearly separable

after normalization. A model without hidden layers effectively performs multinomial logistic regression, which is already sufficient to separate the three flower classes. Adding more layers introduces additional parameters and

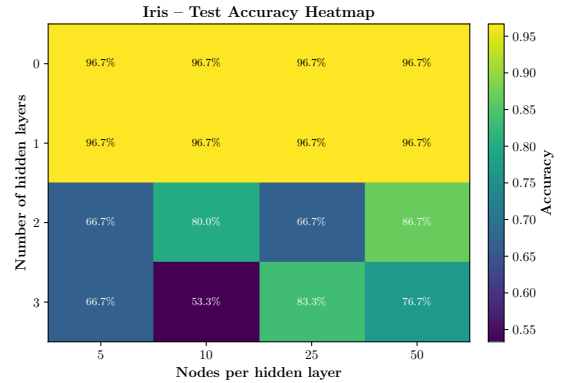


Figure 2: Test accuracy of a fully connected neural network trained on the Iris dataset for different network architectures. The heatmap shows the test accuracy for models with 0–3 hidden layers and 5–50 nodes per hidden layer. A model with no hidden layers achieves the highest accuracy, demonstrating that the Iris dataset is nearly linearly separable. Deeper networks perform worse due to overfitting and unstable convergence on this small dataset.

nonlinearity but provides little benefit for a problem that does not require complex decision boundaries.

The reduced performance for deeper networks can also be explained by the small dataset size and the simple gradient descent training setup. With limited data, deeper models are more prone to overfitting or unstable convergence, especially when using small random initial weights and the same learning rate across all configurations. As a result, shallow networks generalize better on this task, while deeper architectures struggle to optimize effectively and yield lower accuracy.

## IV. CONCLUSION

- State your main findings and interpretations
- Try to discuss the pros and cons of the methods and possible improvements
- State limitations of the study
- Try as far as possible to present perspectives for future work

Som random citations to get the biblio  
As we can see[1] and [2] we can do so and so..

[1] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Pre-*

*diction* (Springer, New York, 2009), 2nd ed., ISBN 978-

0387848570.

- [2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Journal of Machine Learning

Research **12**, 2825 (2011), URL <https://scikit-learn.org/stable/about.html#citing-scikit-learn>.